



← Back to student guides

[📈] MLflow

Tracking & registry

3 levels

111 sections



The Complete Guide

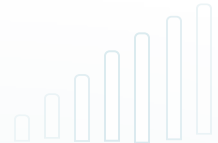
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

RUN

MODEL

REGISTRY

SERVE



Quick Start

A one-page starter for this tool. Scroll through it here, or open it full screen.

Open PDF





MLOps MENA
COMMUNITY

العربية



Join the Community ↗





14

sections

35

examples

This is part two of three. It picks up exactly where Beginner ended and takes **every topic from there further**, then adds the machinery you have not met yet. Nothing is dropped and nothing is repeated for its own sake. Where you already know the basics, we go straight to the depth.

Where this picks up

Topic you already use	What this level adds
Runs	Nested runs, parent/child sweeps, resuming, and bulk operations via the client
Params, metrics, tags	System tags, metric history and steps, batch logging, and search at scale
Autologging	What each integration patches, and how to control or disable it safely
The model format	Custom <code>pyfunc</code> models, wrappers, <code>code_paths</code> , and dependency resolution
Signatures	Explicit schemas, params in signatures, and enforcement behaviour
The registry	A promotion workflow with tags, aliases, approvals, and CI gates
<code>mlflow.evaluate</code>	Custom metrics, validation thresholds, baselines, and static datasets
Datasets	<code>mlflow.data</code> lineage, dataset sources, and what it does and does not prove
Serving	Deployment targets, the deployments API, containers, and scoring-server internals



Projects	Docker environments, backends, and parameterised multi-step flows
Debugging	Client-level diagnosis, artifact-store failures, and reproducing the serving path
new	Tracing & LLM evaluation · plugins · CI/CD gates · model aliases at scale · signature params

Each section starts with the problem it solves, and ends with a **Try it** you can do on a real project in a few minutes.

Runs, nested runs, and the client API

Beginner used one run per script. Real work needs structure, and the client API is what turns MLflow from a logger into something you can automate.

PYTHON

Copy

```
import mlflow

with mlflow.start_run(run_name="sweep: max_depth") as parent:
    mlflow.set_tag("sweep", "max_depth")
    for depth in (3, 5, 8, 12):
        with mlflow.start_run(run_name=f"depth={depth}", nested=True) as child:
            mlflow.log_param("max_depth", depth)
            model = fit(max_depth=depth)
            mlflow.log_metric("val_auc", score(model))
        # The parent records the summary; the children record the detail.
        mlflow.log_metric("best_val_auc", best)
    mlflow.set_tag("best_child_run_id", best_run_id)
```

The parent/child relationship is stored as a tag, `mlflow.parentRunId`, which means it is queryable:



```
children = mlflow.search_runs(  
    experiment_names=["churn"],  
    filter_string=f"tags.mlflow.parentRunId = '{parent.info.run_id}'",  
    order_by=["metrics.val_auc DESC"],  
)
```

The fluent API (`mlflow.log_*`) always writes to the `active` run, which is convenient until it is wrong: in a multi-threaded sweep, or when writing to a run you are not inside. The client API takes an explicit run id and has no global state:

PYTHON

Copy

```
from mlflow import MlflowClient  
client = MlflowClient()  
  
run = client.create_run(experiment_id=exp_id, tags={"team": "risk"}, run_name="explicit")  
client.log_param(run.info.run_id, "max_depth", 5)  
client.log_metric(run.info.run_id, "val_auc", 0.94, step=1)  
client.set_terminated(run.info.run_id, status="FINISHED")
```

Need	Fluent API	Client API
A normal script	✅ <code>mlflow.log_metric(...)</code>	Works, more verbose
Threads or async	Risky: one global active run	✅ explicit run id
Writing to someone else's run	Only via <code>start_run(run_id=...)</code>	✅ direct
Bulk / batch logging	<code>log_metrics</code> , <code>log_params</code>	✅ <code>log_batch</code> for thousands of points
Registry operations	A few module functions	✅ the full surface

batch logging, for a long training loop

Copy



```
now = int(time.time() * 1000)
client.log_batch(
    run_id=run_id,
    metrics=[Metric("loss", v, now, step=i) for i, v in enumerate(losses)],
    params=[Param("optimizer", "adamw")],
    tags=[RunTag("phase", "warmup")],
)
```

Per-step logging over HTTP is a real cost

A separate `log_metric` call per batch on a long run means tens of thousands of HTTP round trips, and it will make your training visibly slower and your tracking server unhappy. Accumulate and use `log_batch`, or log per epoch. This is the most common reason a team's MLflow server appears "slow".

► TRY IT

1. Write a parent run with four nested children and confirm the UI nests them.
2. Query the children with a `tags.mlflow.parentRunId` filter and sort by your metric.
3. Log 5,000 metric points one call at a time, then the same with `log_batch`. Time both.
4. Create a run entirely through the client API, with no active run anywhere.

Expected result – the timing difference in step three is usually an order of magnitude, and it is the single most useful performance fact about MLflow. Step four matters because that is how orchestrators and CI jobs create runs they are not executing inside.

Autologging internals

Beginner said "one line". At this level you have to control what that line does.

Each integration installs **patches** around specific framework functions:



<code>sklearn</code>	<code>fit</code> , <code>fit_predict</code> , <code>fit_transform</code> , and search classes	Params, training metrics, model, child runs per candidate
<code>pytorch</code> (Lightning)	Trainer hooks	Per-epoch metrics, checkpoints, the model
<code>tensorflow</code> / <code>keras</code>	<code>fit</code> , callbacks	Per-epoch metrics, the model, TensorBoard-equivalent scalars
<code>xgboost</code> / <code>lightgbm</code>	<code>train</code>	Params, per-iteration metrics, feature importance, the model
<code>transformers</code>	<code>Trainer</code>	Params, metrics, the pipeline as a model
<code>langchain</code> / <code>openai</code>	Chain and client calls	Traces, inputs and outputs, token counts
<code>spark</code>	Datasource reads	Dataset lineage tags

The options that matter, and every integration accepts them:

PYTHON

Copy

```
mlflow.sklearn.autolog(  
    log_input_examples=False,      # avoid embedding data samples in the model  
    log_model_signatures=True,     # keep signatures on  
    log_models=False,             # you log the model yourself, with registration  
    log_datasets=True,            # dataset lineage where it can be inferred  
    max_tuning_runs=10,           # cap child runs from a search  
    disable=False,  
    exclusive=False,              # True: skip autolog when a run is already active  
    disable_for_unsupported_versions=False,  
    silent=True,                  # suppress the warning noise  
    registered_model_name=None,    # register autologged models  
)
```

Three behaviours worth knowing precisely:



a script where you expected your own run.

`exclusive=True` means "do not interfere". With it, autologging skips logging when a run is already active, which is how you keep a hand-instrumented run from being polluted by framework noise.

`log_input_examples=True` embeds data in the artifact. Useful for documentation, and a data-leak vector if the training data is sensitive, because the example ships inside the model directory that you might publish.

Autolog plus your own `log_model` means two models per run

Autologging logs a model without your signature or your `registered_model_name`. If you also log one explicitly, the run has two model artifacts and nothing states which is authoritative. Set `log_models=False` and log it yourself. That is the pattern to standardise on.

► TRY IT

1. Call `fit()` with autolog on and no active run. Find the run MLflow created for you.
2. Turn on `log_input_examples=True`, log a model, and find the embedded example inside the artifact.
3. Leave autolog's `log_models=True` and also log a model yourself. Look at the resulting artifact tree.
4. Set `exclusive=True` and confirm autologging steps aside inside your own run.

Expected result – step three is the one to see once: two `model/` directories in one run, with different signatures, and no indication which the registry took. That ambiguity is what `log_models=False` exists to prevent.

Custom `pyfunc` models



predict , and it travels as one artifact with one environment.

The problem it solves: your model needs a fitted scaler, a vocabulary, and a threshold. Log only the estimator and every consumer must reimplement the rest, correctly, forever.

model.py

Copy

```
import mlflow
import pandas as pd

class ChurnModel(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        """Runs once at load time. Read anything from context.artifacts."""
        import joblib, json
        self.scaler = joblib.load(context.artifacts["scaler"])
        self.clf = joblib.load(context.artifacts["classifier"])
        with open(context.artifacts["config"]) as handle:
            self.threshold = json.load(handle)["threshold"]

    def predict(self, context, model_input: pd.DataFrame, params=None) → pd.DataFrame:
        threshold = (params or {}).get("threshold", self.threshold)
        features = self.scaler.transform(model_input[self.FEATURES])
        proba = self.clf.predict_proba(features)[:, 1]
        return pd.DataFrame({"proba": proba, "label": (proba ≥ threshold).astype(int)})

FEATURES = ["tenure", "monthly_charges", "support_calls"]
```

log it as one artifact

Copy

```
import mlflow
from mlflow.models import infer_signature, ModelSignature
from mlflow.types.schema import Schema, ColSpec, ParamSchema, ParamSpec

signature = ModelSignature(
    inputs=Schema([ColSpec("double", "tenure"),
                   ColSpec("double", "monthly_charges"),
                   ColSpec("long", "support_calls")]),
    outputs=Schema([ColSpec("double", "proba"), ColSpec("long", "label")]),
    params=ParamSchema([ParamSpec("threshold", "double", 0.5)]),
)
```



```
python_model=ChurnModel(),
artifacts={
    "scaler": "outputs/scaler.pkl",
    "classifier": "outputs/clf.pkl",
    "config": "outputs/config.json",
},
code_paths=["src/features.py"],          # your own modules travel with it
signature=signature,
input_example=X_test.head(3),
pip_requirements=["scikit-learn=1.5.1", "pandas=2.2.2", "joblib=1.4.2"],
registered_model_name="churn-classifier",
)
```

and it loads like any other model

Copy

```
model = mlflow.pyfunc.load_model("models:/churn-classifier@champion")
model.predict(frame)                    # uses the logged threshold
model.predict(frame, params={"threshold": 0.7})  # overridden at call time
```

Piece	Role
<code>load_context</code>	One-time setup; loads the files declared in <code>artifacts</code>
<code>predict(context, input, params)</code>	The whole inference contract, including postprocessing
<code>artifacts={...}</code>	Files copied into the model directory and re-pathed at load
<code>code_paths=[...]</code>	Your modules, copied in so the model does not import your repo
<code>params</code> in the signature	Call-time knobs, validated and documented
<code>pip_requirements</code>	Explicit dependencies: do not let inference guess for a custom model

Without `code_paths`, a custom pyfunc fails at load in any other environment



the serving container with `ModuleNotFoundError` . `code_paths` copies the modules into the artifact, and it is the number-one cause of "works locally, fails in serving" for custom models.

`params` in a signature is underused and excellent

A decision threshold, a top-k, a temperature, anything a caller might want to vary, becomes a validated, documented, call-time parameter rather than a second endpoint or a redeploy. Because it is in the signature, the serving layer exposes and checks it for you.

► TRY IT

1. Wrap a scaler plus a classifier plus a threshold into one `PythonModel` and log it.
2. Load it with `pyfunc.load_model` in a fresh Python process outside your project directory. Note the failure if you omitted `code_paths` .
3. Add `code_paths` and repeat. Then inspect the artifact tree and find your copied module.
4. Add a `threshold` param to the signature and override it at call time.

Expected result – step two is the lesson, and it is worth failing on purpose: the model that loads in your project and dies in a container is almost always a missing `code_paths` . Doing it once makes the error message instantly recognisable.

Dependencies and environments, precisely

Beginner noted that a model carries a `requirements.txt` . At this level you need to know how that list is produced, because it is where portability succeeds or fails.

MLflow infers requirements at log time by **introspecting the loaded modules** and resolving them to installed distributions, then pins them:



```
mlflow==2.16.0
scikit-learn==1.5.1
cloudpickle==3.0.0
numpy==1.26.4
psutil==6.0.0
```

Three ways to take control, in increasing order of explicitness:

PYTHON

Copy

```
# 1. Extra pins on top of inference – for a module inference cannot see
mlflow.sklearn.log_model(model, name="model", extra_pip_requirements=["prometheus-client"])

# 2. Replace inference entirely – for custom pyfunc models, do this
mlflow.pyfunc.log_model(name="model", python_model=ChurnModel(),
                        pip_requirements=["scikit-learn==1.5.1", "pandas==2.2.2"])

# 3. A file, so the pins are reviewable in the repo
mlflow.pyfunc.log_model(name="model", python_model=ChurnModel(),
                        pip_requirements="requirements-serving.txt")
```

Two ways to check the result before it becomes a production problem:

BASH

Copy

```
# Validate that the model loads and scores in a freshly built environment
mlflow models predict -m "runs:/abc123/model" -i input.json \
  --content-type json --env-manager virtualenv

# Print the environment MLflow would build
python -c "import mlflow; print(mlflow.pyfunc.get_model_dependencies('runs:/abc123/model'))"
```

validate a model's signature and environment programmatically

Copy

```
from mlflow.models import validate_serving_input, convert_input_example_to_serving_input
```



Failure	Cause	Fix
<code>ModuleNotFoundError</code> in serving	A module was imported dynamically and never inferred	<code>extra_pip_requirements</code> , or <code>code_paths</code> for your own code
Model loads but predicts differently	A minor version bump changed behaviour	Pin exactly; validate in a built environment
Serving build fails on a private package	The index is not available to the builder	Vendor it, or configure the index in the serving image
A huge slow environment build	Inference pulled in your whole dev environment	Explicit <code>pip_requirements</code> for the serving path only
Works with <code>--env-manager local</code> , fails otherwise	You were testing your own environment, not the logged one	Always validate with <code>virtualenv</code> before shipping

Inference sees imports, not intent

If your code does `importlib.import_module(name)` , or imports inside a branch that did not execute, the dependency is invisible and the model will fail to load elsewhere. For anything custom, declare `pip_requirements` explicitly rather than trusting inference. It takes one line and removes a whole failure class.

► TRY IT

1. Log a model and read its inferred `requirements.txt` . Count how many entries you did not expect.
2. Validate it with `mlflow models predict --env-manager virtualenv` and time the build.
3. Replace inference with an explicit `pip_requirements` list of five packages and validate again. Compare the build time.
4. Use `validate_serving_input` to check the signature contract without standing up a server.

Expected result – the inferred list is usually two or three times longer than necessary, which is both a slow build and a larger attack surface. An explicit serving requirements file is a small change with a visible payoff in step three.



Beginner used the default evaluator. Its real power at this level is **validation**: failing a job when a model is not good enough, which is what turns an evaluation into a gate.

PYTHON

Copy

```
import mlflow
from mlflow.models import MetricThreshold

thresholds = {
    "roc_auc": MetricThreshold(threshold=0.85, greater_is_better=True),
    "log_loss": MetricThreshold(threshold=0.40, greater_is_better=False),
    # Relative to a baseline model, not just an absolute floor
    "f1_score": MetricThreshold(
        threshold=0.80,
        min_absolute_change=0.01,
        min_relative_change=0.01,
        greater_is_better=True,
    ),
}

result = mlflow.evaluate(
    model=candidate_uri,
    data=eval_data,
    targets="label",
    model_type="classifier",
    evaluators=["default"],
    validation_thresholds=thresholds,
    baseline_model=baseline_uri,      # required for relative-change thresholds
)
```

If any threshold fails, `mlflow.evaluate` raises `ModelValidationFailedException`, so in CI the job fails, with the failing metric named.

You can also evaluate a **static dataset** with predictions already computed, which is how you evaluate something MLflow did not train:

PYTHON

Copy



```
validation_thresholds=0.08,  
targets="label",  
predictions="prediction",  
model_type="classifier",  
)
```

Add domain metrics that the default evaluator cannot know about:

PYTHON

Copy

```
from mlflow.models import make_metric  
  
def _cost(eval_df, _builtin):  
    fp = ((eval_df["prediction"] == 1) & (eval_df["target"] == 0)).sum()  
    fn = ((eval_df["prediction"] == 0) & (eval_df["target"] == 1)).sum()  
    return float(50 * fp + 500 * fn)  
  
def _fairness_gap(eval_df, _builtin):  
    grouped = eval_df.groupby(eval_df["segment"])[["prediction"]].mean()  
    return float(grouped.max() - grouped.min())  
  
result = mlflow.evaluate(  
    model=candidate_uri, data=eval_data, targets="label", model_type="classifier",  
    extra_metrics=[  
        make_metric(eval_fn=_cost, greater_is_better=False, name="expected_cost"),  
        make_metric(eval_fn=_fairness_gap, greater_is_better=False, name="fairness_gap")  
    ],  
    validation_thresholds={  
        "expected_cost": MetricThreshold(threshold=20000, greater_is_better=False),  
        "fairness_gap": MetricThreshold(threshold=0.08, greater_is_better=False),  
    },  
)
```

Capability	Why it matters
<code>validation_thresholds</code>	Turns evaluation into a pass/fail gate a pipeline can act on
<code>baseline_model</code>	"Better than what we have" rather than "above an arbitrary number"
<code>min_relative_change</code>	Blocks promotions that are within noise



<code>extra_metrics</code>	business cost and fairness become gate conditions, not slides
Static datasets	Evaluate any predictions, including from a non-MLflow model

A relative threshold against a baseline is the honest gate

An absolute floor passes forever once you clear it. Comparing against the current champion with a minimum change means a promotion has to be an improvement, which is what stops a quarter of churn from noise-level "wins".

► TRY IT

1. Add `validation_thresholds` that your model passes, then tighten one until it raises.
2. Pass the current champion as `baseline_model` with a `min_relative_change` and confirm a marginal model is rejected.
3. Add a cost metric and gate on it. Note whether it changes which model you would ship.
4. Evaluate a static prediction column with no model at all.

Expected result – step two is the one that changes process rather than code: with a relative gate in place, "we improved AUC by 0.0004" stops being a promotion, and the argument moves to whether the change is real.

The registry as a promotion workflow

Beginner registered versions and moved an alias. Most of the operational value lives in the workflow around it.

promote.py

Copy

```
import mlflow
from mlflow import MlflowClient
from mlflow.models import MetricThreshold
```



```
def metric(version, key="roc_auc"):
    return client.get_run(version.run_id).data.metrics.get(key, 0.0)

candidates = client.search_model_versions(
    f"name = '{NAME}' and tags.validated = 'true'"
)
best = max(candidates, key=metric)

try:
    champion = client.get_model_version_by_alias(NAME, "champion")
except mlflow.exceptions.MlflowException:
    champion = None

# Compare on a held-out dataset, not on the training run's own metric
result = mlflow.evaluate(
    model=f"models://{NAME}/{best.version}",
    data=holdout, targets="label", model_type="classifier",
    baseline_model=(f"models://{NAME}/{champion.version}" if champion else None),
    validation_thresholds={
        "roc_auc": MetricThreshold(threshold=0.85, min_relative_change=0.01,
                                    greater_is_better=True),
    },
) # raises ModelValidationFailedException on failure

client.set_model_version_tag(NAME, best.version, "holdout_auc", f"{result.metrics['roc_auc']:.2f}")
client.set_model_version_tag(NAME, best.version, "approved_by", "ci")
if champion:
    client.set_registered_model_alias(NAME, "previous", champion.version)
client.set_registered_model_alias(NAME, "champion", best.version)
```

1 Register

Training logs and registers a version, tagged `validated=false` .

2 Validate

An evaluation job runs thresholds on a holdout set and tags `validated=true` .

3 Compare

Against the current champion, with a minimum relative change.



5 Roll back

Move `champion` back to `previous` . One call, no redeploy.

Alias	Convention
<code>champion</code>	What production serves
<code>challenger</code>	Under evaluation, possibly shadow-scored
<code>previous</code>	The immediate rollback target
<code>baseline</code>	A fixed reference for relative comparisons

Rollback is only one call if you kept a `previous` alias

Without it, rolling back means finding the last good version number in a hurry, from a UI, during an incident. Setting `previous` at promotion time costs one line and turns rollback into a known, rehearsable action.

Searching the registry is the same filter language, which is what makes governance scriptable:

PYTHON

Copy

```
client.search_registered_models(filter_string="name LIKE 'churn%')
client.search_model_versions(f"name = '{NAME}' and tags.approved_by = 'ci'")
```

► TRY IT

1. Tag a version `validated=true` only after a threshold evaluation passes, and make your promotion script require that tag.
2. Promote, keeping `previous` , then roll back with one call and confirm your serving endpoint follows.
3. Search model versions by tag and confirm you can list what CI has approved.



rollback you have performed once takes thirty seconds; one you have only read about takes an incident.

Dataset tracking and lineage

Beginner called `log_input` once. The call records more than it proves.

PYTHON

Copy

```
import mlflow.data
import pandas as pd

frame = pd.read_parquet("s3://bucket/data/train_v3.parquet")

dataset = mlflow.data.from_pandas(
    frame,
    source="s3://bucket/data/train_v3.parquet",
    name="churn-train",
    targets="label",
    digest=None,          # computed for you: a content hash of the frame
)

with mlflow.start_run():
    mlflow.log_input(dataset, context="training", tags={"version": "v3"})
    mlflow.log_input(eval_dataset, context="evaluation")
```

Constructor	Source type
<code>from_pandas</code>	A DataFrame, with a declared source URI
<code>from_numpy</code>	Arrays
<code>from_spark</code> / <code>load_delta</code>	Spark tables and Delta versions: the strongest lineage available



<code>from_nuggingrace_dataset</code>	HF datasets, with their own revision
<code>mlflow.data.meta_dataset</code>	A pointer only, when the data is too large to hash

What a logged dataset records: a **name**, a **source URI**, a **digest** (a content hash of the profiled data), a **schema**, and an optional **profile** such as row and column counts. What that gives you is a run that names its input and can be compared against another run's input by digest.

What it does **not** give you: immutability. The digest proves the frame you profiled, not that the object at `s3://bucket/data/train_v3.parquet` is unchanged today. If someone overwrites that path, the run's recorded source now points at different bytes and nothing in MLflow notices.

A source URI is a name, not a guarantee

MLflow's dataset tracking is **lineage annotation**, not data versioning. It tells you which path a run read and what that data looked like at the time. Real immutability comes from the storage side: versioned buckets, dated immutable prefixes, or a table format with time travel. If you need "rebuild exactly this dataset", pair MLflow with something that provides it.

The pattern that closes most of the gap without another tool:

PYTHON

Copy

```
# Dated, immutable prefixes plus the digest recorded on the run
SOURCE = "s3://bucket/data/churn/dt=2026-08-01/train.parquet" # never overwritten
dataset = mlflow.data.from_pandas(frame, source=SOURCE, name="churn-train", targets="lab

with mlflow.start_run():
    mlflow.log_input(dataset, context="training")
    mlflow.log_params({"data_partition": "dt=2026-08-01"}) # also a searchable param
```

► TRY IT



3. Overwrite the file at the source URI and rerun. Confirm nothing warns you.

4. Switch to a dated immutable prefix and log the partition as a param as well.

Expected result – step three is the important one: the digest changes, but nothing connects the old run to the fact that its source moved underneath it. Knowing that boundary is what stops you claiming more reproducibility than you have.

Deployment targets and the deployments API

Beginner served locally. At this level deployment is a target, and MLflow abstracts several behind one interface.

BASH

Copy

```
# Built-in local scoring server
mlflow models serve -m "models:/churn-classifier@champion" --port 5001 --workers 4

# A container to hand to a platform team
mlflow models build-docker -m "models:/churn-classifier@champion" \
  -n churn-scorer --enable-mlserver

# Managed targets via plugins
mlflow deployments create -t sagemaker --name churn -m "models:/churn-classifier@champion" \
  -C region_name=eu-west-1 -C instance_type=ml.m5.large
mlflow deployments list -t sagemaker
mlflow deployments update -t sagemaker --name churn -m "models:/churn-classifier@champion"
mlflow deployments delete -t sagemaker --name churn
```

the same thing from Python

Copy

```
from mlflow.deployments import get_deploy_client
```



```
client.predict(deployment_name="churn", inputs=payload)
```

Target	Comes from
Local scoring server	Built in
Docker image	Built in (build-docker)
Spark UDF	mlflow.pyfunc.spark_udf : batch scoring at scale
SageMaker / Azure ML / Databricks	Plugins or vendor integrations
Kubernetes	The container, plus your own manifests

Batch scoring in Spark deserves its own mention, because it is where the pyfunc contract pays off most:

PYTHON

Copy

```
from pyspark.sql.functions import struct
import mlflow.pyfunc

predict = mlflow.pyfunc.spark_udf(spark, "models:/churn-classifier@champion", result_type=
scored = df.withColumn("score", predict(struct(*df.columns)))
```

Two serving internals worth knowing:

Workers matter. The default scoring server runs a single worker; `--workers N` uses more. For real throughput, `--enable-mlserver` swaps in MLServer, which supports adaptive batching and is the better base for a container.

The `/invocations` contract is fixed. Four payload shapes (`dataframe_split` , `dataframe_records` , `instances` , `inputs`) plus `params` for signature params. Anything calling your endpoint is coding against that contract, so a signature change is a breaking API change.



contract. Because promotion moves an alias with no deploy step, a signature change can reach production without anyone reviewing an API diff. Treat the signature as a public interface: additive changes only, or a new registered model name.

► TRY IT

1. Serve with `--workers 1` and then `--workers 4`, and compare throughput with a simple load loop.
2. Build a Docker image with `--enable-mlserver` and call it.
3. Score a DataFrame with `spark_udf` if you have Spark available, or read the API and note that no framework code is involved.
4. Rename a feature column, relog, promote, and call the old endpoint payload. Read the failure.

Expected result – step four is the one to internalise: a promotion that silently breaks every caller is possible precisely because aliases decouple deployment from code review. That is the risk that comes with the convenience.

MLflow Tracing and LLM evaluation

Newer MLflow versions add tracing and GenAI evaluation, and they are increasingly what interviews ask about.

Tracing records the internal steps of a call (a chain, a retrieval, a tool use, a model call) as a tree of spans with inputs, outputs, latency, and token counts.

PYTHON

Copy

```
import mlflow

mlflow.set_experiment("rag-assistant")
mlflow.langchain.autolog()          # traces every chain invocation
# also: mlflow.openai.autolog(), mlflow.llama_index.autolog(), and others
```



```
return vector_store.similarity_search(question, k=k)
```

```
@mlflow.trace(span_type="LLM")
def answer(question: str):
    docs = retrieve(question)
    return llm.invoke(build_prompt(question, docs))

with mlflow.start_run():
    answer("What is our refund window?")
```

manual spans, when a decorator does not fit

Copy

```
with mlflow.start_span(name="rerank", span_type="RERANKER") as span:
    span.set_inputs({"candidates": len(docs)})
    ranked = reranker.rank(docs)
    span.set_outputs({"kept": len(ranked)})
    span.set_attribute("model", "bge-reranker-v2")
```

Concept	Meaning
Trace	One end-to-end request
Span	One step inside it, with a type, inputs, outputs, and timing
<code>span_type</code>	<code>LLM</code> , <code>RETRIEVER</code> , <code>RERANKER</code> , <code>TOOL</code> , <code>CHAIN</code> , <code>PARSER</code> , ...
Attributes	Arbitrary metadata: model name, token counts, cost
Autolog integrations	Trace popular frameworks without writing spans yourself

LLM evaluation extends `mlflow.evaluate` with text metrics, including LLM-as-a-judge:

PYTHON

Copy

```
from mlflow.metrics.genai import answer_correctness, faithfulness, answer_relevance

eval_df = pd.DataFrame({
    "inputs": questions,
```



```
result = mlflow.evaluate(  
    model=my_qa_function,          # any callable, or a pyfunc URI  
    data=eval_df,  
    targets="ground_truth",  
    model_type="question-answering",  
    extra_metrics=[  
        answer_correctness(model="openai:/gpt-4o-mini"),  
        faithfulness(model="openai:/gpt-4o-mini"),  
        answer_relevance(model="openai:/gpt-4o-mini"),  
    ],  
    evaluator_config={"col_mapping": {"context": "context"}},  
)
```

Traces contain the prompt and the response: treat them as data

A trace stores inputs and outputs verbatim. For a customer-facing assistant that means user messages, and potentially personal data, sitting in your tracking backend and in your backups. Decide on redaction and retention before you enable tracing on production traffic, not after.

LLM-as-a-judge metrics cost money and vary

Every judged row is an API call, so a 5,000-row evaluation has a real bill and a real latency. Judges are also non-deterministic, so pin the judge model, fix its version, and report the judged metric with the judge named, or you cannot compare two evaluations.

► TRY IT

1. Decorate two functions with `@mlflow.trace` and inspect the span tree in the UI.
2. Add an attribute with token counts and find it on the span.
3. Run an LLM evaluation on twenty rows with one judged metric and note the cost and duration.
4. Look at what a trace stored from a realistic prompt, and decide what you would redact.

Expected result – step four is the one people skip. Reading your own trace with real input in it is what makes the retention and redaction question concrete rather than theoretical.



The pattern worth building: a pull request that changes training code produces a tracked run, evaluates it against thresholds and the current champion, and fails the check if it regresses. That makes model quality a reviewable property of the diff.

.github/workflows/train.yml

Copy

```
name: train
on:
  pull_request:
    paths: ['src/**', 'MLproject', 'requirements.txt', 'python_env.yaml']

permissions:
  contents: read

jobs:
  evaluate:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: pip

      - run: pip install -r requirements.txt

      - name: Train and gate
        env:
          MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_TRACKING_URI }
          MLFLOW_TRACKING_USERNAME: ${ secrets.MLFLOW_TRACKING_USERNAME }
          MLFLOW_TRACKING_PASSWORD: ${ secrets.MLFLOW_TRACKING_PASSWORD }
          AWS_ROLE_ARN: ${ secrets.AWS_ROLE_ARN }
        run: python ci/train_and_gate.py --experiment "churn/ci"
```

ci/train_and_gate.py

Copy



```
from mlflow import MlflowClient
from mlflow.models import MetricThreshold

ap = argparse.ArgumentParser()
ap.add_argument("--experiment", required=True)
ap.add_argument("--name", default="churn-classifier")
args = ap.parse_args()

sha = os.environ.get("GITHUB_SHA", "local")[:8]
pr = os.environ.get("GITHUB_REF_NAME", "local")

mlflow.set_experiment(args.experiment)
with mlflow.start_run(run_name=f"pr {pr} @ {sha}",
                      tags={"ci": "true", "pr": pr, "commit": sha}) as run:
    model_info = train_and_log()          # your training, returning the ModelInfo

client = MlflowClient()
try:
    champion = client.get_model_version_by_alias(args.name, "champion")
    baseline = f"models/{args.name}/{champion.version}"
except mlflow.exceptions.MlflowException:
    baseline = None

try:
    result = mlflow.evaluate(
        model=model_info.model_uri,
        data=load_holdout(), targets="label", model_type="classifier",
        baseline_model=baseline,
        validation_thresholds={
            "roc_auc": MetricThreshold(threshold=0.85, min_relative_change=0.005,
                                       greater_is_better=True),
            "expected_cost": MetricThreshold(threshold=20000, greater_is_better=False),
        },
    )
except mlflow.exceptions.MlflowException as exc:
    print(f"::error::validation failed - {exc}")
    sys.exit(1)

url = f"{mlflow.get_tracking_uri()}/#/experiments/{run.info.experiment_id}/runs/{run.info.run_id}"
print(f"::notice::roc_auc={result.metrics['roc_auc']:.4f} - {url}")
```



A separate <code>*/ci</code> experiment	keeps pull-request runs out of the research table, and easy to clean up
Tags for <code>ci</code> , <code>pr</code> , and <code>commit</code>	Every CI run is filterable and deletable as a group
<code>validation_thresholds</code> against the champion	The gate is "better than production", not "above a number"
Credentials as secrets in env vars	No config file to leak; one place to rotate
A short training budget	The gate answers "did this regress", not "is this the best model"
The run URL in the log	A reviewer gets one click to the full run
A job timeout	An unreachable tracking server should not burn thirty minutes

CI credentials can write to your registry: scope them

A tracking credential that can register models and move aliases is a production-changing credential. On a public repository, a fork pull request holding it means a stranger can promote a model. Use a write-restricted credential for pull requests that can create runs but not move aliases, and keep promotion on a protected-branch workflow.

► TRY IT

1. Run the gate script locally against your own tracking server first.
2. Wire it into CI and open a pull request that harms the metric. Confirm the check fails and names the metric.
3. Open one that improves it marginally and confirm the relative threshold still rejects it.
4. Point the tracking URI at an unreachable host and confirm your timeout, not the runner limit, ends the job.

Expected result – step three is the valuable one: a gate that rejects noise-level improvements changes what the team argues about, from "the number went up" to "is this difference real".



Beginner had a tab order. When the UI runs out, work through these.

01 Separate the two stores in your head

Metadata failures are backend-store problems (database, permissions, migrations). Missing files are artifact-store problems (credentials, URI, network). The symptoms are different.

02 Read the artifact URI on the run, not the config

`run.info.artifact_uri` is what that run used. A run created before a config change points at the old location forever.

03 Reproduce the serving path, not your notebook

`mlflow models predict --env-manager virtualenv` exercises what a container would do, with a readable traceback.

04 Check the client and server versions

A newer client can send fields an older server rejects. `mlflow.__version__` against the server's footer.

05 Turn up logging

`MLFLOW_TRACKING_URI` plus Python logging at DEBUG on `mlflow` shows every REST call, which is how you diagnose a hang or a 403.

06 Compare a good run against a bad one from the client

Params, tags, and the model's requirements, and the diff is usually one line long.

BASH

Copy

```
export MLFLOW_TRACKING_URI=http://mlflow.internal:5000
python - <<'PY'
import logging, mlflow
logging.basicConfig(level=logging.DEBUG)
logging.getLogger("mlflow").setLevel(logging.DEBUG)
```



```
# Is the server even there, and what version?
curl -s "$MLFLOW_TRACKING_URI/health"
curl -s "$MLFLOW_TRACKING_URI/api/2.0/mlflow/experiments/search?max_results=1"
```

compare two runs programmatically

Copy

```
from mlflow import MlflowClient
client = MlflowClient()

good, bad = client.get_run("...good..."), client.get_run("...bad...")
for key in sorted(set(good.data.params) | set(bad.data.params)):
    if good.data.params.get(key) != bad.data.params.get(key):
        print(f"param {key}: {good.data.params.get(key)!r} → {bad.data.params.get(key)!r}")

print("commit:", good.data.tags.get("mlflow.source.git.commit"),
      "→", bad.data.tags.get("mlflow.source.git.commit"))
print("artifacts:", good.info.artifact_uri, "→", bad.info.artifact_uri)
```

Symptom	Cause	Fix
403 or 401 from the client	Auth on the tracking server, no credentials set	<code>MLFLOW_TRACKING_USERNAME</code> / <code>PASSWORD</code> , or a token
Metadata writes fine, artifacts fail	Client lacks object-store credentials	The artifact store is accessed by the client , not proxied by default
Old runs' artifacts unreachable	Artifact root changed after they were created	The URI is stored per run; migrate or keep the old location readable
<code>INVALID_PARAMETER_VALUE</code> on a new field	Client newer than server	Align versions
Registry operations fail, tracking works	Backend store is file-based	Database backend
Serving works locally, not in the image	<code>--env-manager local</code> masked a mismatch	Validate with <code>virtualenv</code>



which means every user and every CI job needs object-store credentials. That is the single most surprising piece of MLflow's architecture, and it is why "metrics appear but artifacts 404" is so common. Proxied artifact access exists, Senior covers it, and it changes the whole credential story.

► TRY IT

1. Enable DEBUG logging and read the REST calls a single `log_metric` makes.
2. Remove your object-store credentials and confirm metrics still log while artifacts fail.
3. Change the artifact root on the server, then check an old run's `artifact_uri` and see that it is unchanged.
4. Diff a working and a broken run with the script above.

Expected result – step two is the architecture lesson in thirty seconds. Once you have seen metrics succeed while artifacts fail, the direct-client-access model is obvious and the whole class of errors becomes readable.

Putting it all together

A complete project using everything on this page. Nothing here is new. Read it as a whole and you should be able to justify every line.

project layout

Copy

```
.
├── .github/workflows/train.yml      # PR gate: run, evaluate, threshold
├── MLproject                       # entry points: main, evaluate, promote
├── python_env.yml
├── requirements.txt                # pinned, includes mlflow==
├── requirements-serving.txt        # the smaller explicit serving env
├── src/
│   └── features.py                 # travels with the model via code_paths
```



```
| | promote.py           # tags, previous, champion
| | batch_score.py       # spark_udf or pyfunc.load_model
| ci/train_and_gate.py
```

src/model.py

Copy

```
import mlflow
import pandas as pd
from src.features import build_features      # copied in via code_paths

class ChurnModel(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        import joblib, json
        self.scaler = joblib.load(context.artifacts["scaler"])
        self.clf = joblib.load(context.artifacts["classifier"])
        with open(context.artifacts["config"]) as handle:
            self.threshold = json.load(handle)["threshold"]

    def predict(self, context, model_input: pd.DataFrame, params=None) → pd.DataFrame:
        threshold = (params or {}).get("threshold", self.threshold)
        features = self.scaler.transform(build_features(model_input))
        proba = self.clf.predict_proba(features)[: , 1]
        return pd.DataFrame({"proba": proba, "label": (proba ≥ threshold).astype(int)})
```

src/train.py

Copy

```
import argparse
import mlflow
import mlflow.data
from mlflow.models import ModelSignature
from mlflow.types.schema import Schema, ColSpec, ParamSchema, ParamSpec
from src.model import ChurnModel

ap = argparse.ArgumentParser()
ap.add_argument("--depths", type=str, default="3,5,8,12")
ap.add_argument("--data-partition", default="dt=2026-08-01")
ap.add_argument("--seed", type=int, default=42)
args = ap.parse_args()

mlflow.set_experiment("churn")
```



```
frame = read_parquet(SOURCE)
dataset = mlflow.data.from_pandas(frame, source=SOURCE, name="churn-train", targets="lab

signature = ModelSignature(                                                    # 3
    inputs=Schema([ColSpec("double", "tenure"),
                    ColSpec("double", "monthly_charges"),
                    ColSpec("long", "support_calls")]),
    outputs=Schema([ColSpec("double", "proba"), ColSpec("long", "label")]),
    params=ParamSchema([ParamSpec("threshold", "double", 0.5)]),
)

with mlflow.start_run(run_name=f"sweep {args.data_partition}",                  # 4
                     tags={"team": "risk", "sweep": "max_depth"}) as parent:
    mlflow.log_input(dataset, context="training")
    mlflow.log_params({"seed": args.seed, "data_partition": args.data_partition})

best = None
for depth in [int(d) for d in args.depths.split(",")]:
    with mlflow.start_run(run_name=f"depth={depth}", nested=True) as child:    # 5
        model, metrics = fit(frame, max_depth=depth, seed=args.seed)
        mlflow.log_metrics(metrics)                                           # 6 b
        if best is None or metrics["val_auc"] > best[1]:
            best = (child.info.run_id, metrics["val_auc"], model)

run_id, auc, model = best
mlflow.log_metric("best_val_auc", auc)
mlflow.set_tag("best_child_run_id", run_id)

info = mlflow.pyfunc.log_model(                                              # 7
    name="model",
    python_model=ChurnModel(),
    artifacts={"scaler": "outputs/scaler.pkl",
               "classifier": "outputs/clf.pkl",
               "config": "outputs/config.json"},
    code_paths=["src/features.py", "src/model.py"],                          # 8
    signature=signature,
    input_example=frame.head(3).drop(columns=["label"]),
    pip_requirements="requirements-serving.txt",                             # 9
    registered_model_name="churn-classifier",
)
print(info.model_uri)
```



```
import mlflow
from mlflow import MlflowClient
from mlflow.models import MetricThreshold, make_metric

NAME = "churn-classifier"
client = MlflowClient()
version = client.get_latest_versions(NAME)[0] # or a passed-in version

def _cost(eval_df, _builtin):
    fp = ((eval_df["prediction"] == 1) & (eval_df["target"] == 0)).sum()
    fn = ((eval_df["prediction"] == 0) & (eval_df["target"] == 1)).sum()
    return float(50 * fp + 500 * fn)

try:
    champion = client.get_model_version_by_alias(NAME, "champion")
    baseline = f"models:{NAME}/{champion.version}"
except mlflow.exceptions.MlflowException:
    baseline = None

with mlflow.start_run(run_name=f"validate v{version.version}", tags={"purpose": "validation"},
                    result = mlflow.evaluate( # 10
                        model=f"models:{NAME}/{version.version}",
                        data=load_holdout(), targets="label", model_type="classifier",
                        baseline_model=baseline,
                        extra_metrics=[make_metric(eval_fn=_cost, greater_is_better=False, name="expected_validation_thresholds={
                            "roc_auc": MetricThreshold(threshold=0.85, min_relative_change=0.005,
                                                        greater_is_better=True),
                            "expected_cost": MetricThreshold(threshold=20000, greater_is_better=False),
                        }],
                    )

client.set_model_version_tag(NAME, version.version, "validated", "true") # 11
client.set_model_version_tag(NAME, version.version, "holdout_auc", f"{result.metrics['roc_auc']}
```

src/promote.py

Copy

```
import mlflow
from mlflow import MlflowClient

NAME = "churn-classifier"
client = MlflowClient()
```



```
try:
    champion = client.get_model_version_by_alias(NAME, "champion")
    client.set_registered_model_alias(NAME, "previous", champion.version) # 12
except mlflow.exceptions.MlflowException:
    pass

client.set_registered_model_alias(NAME, "champion", best.version)
print("champion is now version", best.version)
```

BASH

Copy

```
# The everyday loop
mlflow run . -P depths=3,5,8,12 -P data_partition=dt=2026-08-01
mlflow run . -e evaluate
mlflow run . -e promote

# Validate the serving contract before anyone depends on it
mlflow models predict -m "models:/churn-classifier@champion" -i holdout.json \
    --content-type json --env-manager virtualenv

# Then serve, or hand over a container
mlflow models build-docker -m "models:/churn-classifier@champion" -n churn-scorer --enab
```

Twelve decisions in there are the whole lesson of this page:

Decision	Section
Autolog with <code>log_models=False</code> and no input examples	Autologging internals
A dated immutable source, logged as a dataset and a param	Dataset tracking and lineage
An explicit signature, including a <code>params</code> schema	Custom pyfunc models
A named parent run with sweep tags	Runs and nested runs
Nested children, one per candidate	Runs and nested runs
Batched metric logging	Runs and nested runs



A custom pyfunc that carries preprocessing and the threshold	Custom pyfunc models
<code>code_paths</code> so the model imports nothing from your repo	Custom pyfunc models
An explicit, smaller serving requirements file	Dependencies and environments
Thresholds relative to the champion, plus a cost metric	<code>mlflow.evaluate</code> with thresholds
<code>validated=true</code> tagged only after the gate passes	The registry as a promotion workflow
A <code>previous</code> alias set at promotion, so rollback is one call	The registry as a promotion workflow

► TRY IT: THE ONE THAT MATTERS

1. Take this layout into a real project. Get the sweep producing a parent with nested children.
2. Wrap your preprocessing into a custom pyfunc with `code_paths` , and validate it loads in a fresh environment outside the project directory.
3. Add threshold validation against the current champion and confirm a marginal model is rejected.
4. Promote with a `previous` alias, then roll back in one call while an endpoint is live.
5. Break the signature on purpose, promote, and watch a caller fail, then decide your team's rule for signature changes.

Expected result – step five is the acceptance test for the whole page. If a signature change can reach production without review, that is a process gap rather than a tooling one, and finding it deliberately is much cheaper than finding it in an incident.

Where you are now

You can structure sweeps as nested runs and drive MLflow entirely from the client API, control what autologging does instead of accepting it, wrap arbitrary preprocessing and postprocessing into a portable custom pyfunc with its own code and dependencies, declare



registry as a workflow with validation tags and a rollback alias, record dataset lineage and state its limits, deploy to several targets including Spark and containers, trace and evaluate LLM applications, and gate a pull request on model quality.

Can you...	
Say when to use the client API over the fluent one?	Threads, other runs, bulk, registry work
Explain why per-step HTTP logging is slow?	One round trip per point; use <code>log_batch</code>
Say what autolog does about models by default?	Logs one without your signature or registration
Explain why a custom pyfunc needs <code>code_paths</code> ?	It pickles by class reference; the module must be importable
Say what <code>params</code> in a signature buys you?	Validated call-time knobs with no redeploy
Name how requirements are inferred, and how it fails?	Module introspection; dynamic imports are invisible
Give the command that reproduces the serving path?	<code>mlflow models predict --env-manager virtualenv</code>
Explain a relative validation threshold?	Better than the baseline by a minimum margin
Say what dataset logging does <code>not</code> prove?	Immutability: a source URI is a name, not a guarantee
Name the rollback mechanism?	Move <code>champion</code> to <code>previous</code> ; one call
Say why signature changes are dangerous with aliases?	Promotion is not a code review, but it is an API change
Name MLflow's most surprising architectural detail?	Clients read and write the artifact store directly



tenancy across teams, credential and access design, database scaling and what breaks first, backup and upgrade procedure, artifact cost and retention policy, lineage and audit that satisfies a reviewer, approval trails and separation of duties on promotion, incident playbooks, and where MLflow stops and a feature store, an orchestrator, or a dedicated serving platform begins.



PREVIOUS

Beginner · Tips & Tricks

NEXT UP

Mid-level · Interview Review



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



Brainsmingle space

Free to join this week with code **FREECORETEAM**